

Exercícios — Aula 14 — Grafos simples e DFS

Técnicas de Programação - 2026/02

Última atualização: July 26, 2026

Instruções

- Leia o pseudocódigo antes de executar mentalmente.
- Em DFS, acompanhe o vértice atual, o vetor `visitado` e a ordem das chamadas.
- Quando o grafo for uma matriz, pense nas células livres como vértices e nos deslocamentos como arestas implícitas.

Exercício 1 — Simulação de DFS em lista de adjacência

Considere o grafo não direcionado abaixo.

```
adj[0] = [1, 2]
adj[1] = [0, 3]
adj[2] = [0, 4]
adj[3] = [1, 4]
adj[4] = [2, 3]
adj[5] = []
```

Simule `DFS(adj, 0, visitado)` usando a ordem dos vizinhos como aparece na lista.

Algorithm 1: Dfs

Result: Executa DFS.

Input : list de adjacência `adj`, int `atual`, array `visitado`

Output: none

```
visitado[atual] ← true
IMPRIMIR(atual)
foreach vizinho ∈ adj[atual] do
    if visitado[vizinho] = false then
        DFS(adj, vizinho, visitado)
    end
end
end
```

Preencha a tabela.

	passo	chamada ativa	vizinho analisado	nova chamada?	visitados ao final do passo	saída acumulada
	1	DFS(0)	-	-		
	2					
	3					
	4					
	5					

O vértice 5 é visitado? Por quê?

Exercício 2 — Montar lista de adjacência

Monte a lista de adjacência de um grafo não direcionado com vértices 0, 1, 2, 3, 4, 5 e arestas:

(0, 1), (0, 2), (1, 3), (2, 3), (2, 4), (4, 5)

Preencha:

```
adj[0] = [...]  
adj[1] = [...]  
adj[2] = [...]  
adj[3] = [...]  
adj[4] = [...]  
adj[5] = [...]
```

Depois responda:

1. Se o grafo fosse direcionado, o que mudaria?
2. Qual vértice tem mais vizinhos?
3. Quantas entradas aparecem no total nas listas de um grafo não direcionado?

Exercício 3 — Chamadas válidas em matriz

No labirinto abaixo, # é parede, . é livre, C é a célula atual e x indica célula já visitada.

```
#####
#..#..#
#.C...#
#xx#..#
#####
```

Use vizinhança de 4 direções: cima, baixo, esquerda e direita.

Preencha a tabela.

direção	coordenada vizinha	dentro da matriz?	é parede?	já visitada?	faria chamada DFS?
cima					
baixo					
esquerda					
direita					

Explique a regra que decide se uma chamada recursiva deve ser feita.

Exercício 4 — Existe caminho

Escreva pseudocódigo para EXISTE-CAMINHO.

Contrato:

- entrada: lista de adjacência **adj**, origem **origem**, destino **destino**;
- saída: **VERDADEIRO** se existe caminho, **FALSO** caso contrário;
- use DFS recursiva;
- use um array **visitado** para evitar ciclos.

Esqueleto sugerido:

Algorithm 2: ExisteCaminho

Result: Executa EXISTE-CAMINHO.

Input : adj, origem, destino

Output: boolean

visitado $\leftarrow$ ARRAY-DE-FALSOS(TAMANHO(*adj*))
return DFS-CAMINHO(*adj*, *origem*, *destino*, *visitado*)

Algorithm 3: DfsCaminho

Result: Executa DFS-CAMINHO.

Input : adj, atual, destino, visitado

Output: boolean

...

Teste sua ideia no grafo do Exercício 1 para:

- origem 0, destino 4;
- origem 0, destino 5;
- origem 3, destino 2.

Exercício 5 — Contar componentes

Complete o pseudocódigo para contar componentes conectadas.

Algorithm 4: ContarComponentes

Result: Executa CONTAR-COMPONENTES.

Input : list de adjacência *adj*

Output: int

```
visitado ← ARRAY-DE-FALSOS(TAMANHO(adj))  
componentes ← 0  
for v ← 0 to TAMANHO(adj) − 1 do  
    if _____ then  
        DFS(adj, v, visitado)  
        componentes ← _____  
    end  
end  
return componentes
```

Use o grafo do Exercício 1 e diga:

1. Quantas componentes existem?
2. Em quais valores de *v* o contador aumenta?
3. Por que não devemos incrementar o contador para vértices já visitados?

Exercício 6 — Grafo explícito ou implícito

Classifique cada situação como grafo explícito ou grafo implícito.

Situação	Explícito ou implícito?	Como obter vizinhos?
Lista de salas e portas entre salas		
Labirinto em matriz com paredes		
Rede social com lista de amizades		
Tabuleiro em que uma peça pode mover para casas válidas		
Mapa de estradas representado por pares de cidades conectadas		

Depois, responda:

1. Em qual representação as arestas já estão guardadas?
2. Em qual representação os vizinhos são calculados quando precisamos deles?
3. O papel de `visitado` muda entre as duas representações?

Créditos e reaproveitamento

Exercícios adaptados de marcação de vizinhos, critérios de parada e labirintos dos handouts antigos devem indicar:

Adaptado de material de Igor Montagner para a disciplina Técnicas de Programação.

Fonte externa opcional para variações conceituais: Princeton Undirected Graphs, <https://algs4.cs.princeton.edu/41graph/>.